CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

UNIVERSITY INSTITUTE OF COMPUTING

PROJECT REPORT
ON
SMART URBAN WEBSITE
Smart Urban Route Planning System with C++

OBJECT ORIENTED PROGRAMMING

25CAH-201

SUBMITTED BY

Name: S. S. Sahil

UID: 25BCD10109

Branch: BCA (Data Science)

Section: 25BCD2(B)

Semester: 3

SUBMITTED TO

Name: MR. JITENDRA KUMAR

Designation: Professor

Date of Submission: 01/11/2026

Signature: _____

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

ACKNOWLEDGEMENT

I would like to express my sincere thanks to Chandigarh University and the University Institute of Computing (UIC) for giving me the opportunity to work on this project as part of my BCA (Data Science) course.

I am grateful to my subject teacher for C++ Object-Oriented Programming for the guidance and the practical knowledge shared during the semester, which helped me understand the concepts that were needed to build this project.

I also thank the creator of the cpp-httplib library, whose open-source work made it possible to run a small web server from a C++ program without much extra setup. Lastly, I thank my family and friends for their support and encouragement while I was building and testing the project.

S S SAHIL

UID: 25BCD10109

ABSTRACT

Cities depend on road networks, and travel time on these roads changes with traffic. This project, the Smart Urban Website, is a small web application written in C++17 that finds the fastest route between two locations in a simulated city. The city is modelled as a weighted graph of 8 locations and 12 roads. Each road stores its distance in kilometres, its normal travel time in minutes, a traffic multiplier and an open or closed status.

The route is calculated with Dijkstra's shortest-path algorithm, where the weight of a road is its normal travel time multiplied by the current traffic multiplier. Closed roads are skipped, so the program can find a different route when traffic changes. The C++ program also works as a web server using the cpp-httplib library. It serves a single page built with HTML, CSS and JavaScript that draws the city as an interactive SVG map and shows the chosen route, total distance, estimated time and road conditions. The page talks to the server through four simple GET API endpoints, and the shared data is protected with a mutex.

The project was built, run and tested on a Linux system using g++. The tests covered route calculation, traffic simulation, reset and invalid input handling, and the results are given in Chapter 4. The report also records the limitations that were found, for example that the program has no database, uses a fixed map, and that the traffic simulation applies only its first scenario in the submitted code. The project shows how basic data structures, graph algorithms and C++ features can be combined into a working smart-city style application.

Keywords: C++17, Dijkstra's algorithm, route planning, traffic simulation, cpp-httplib, REST API, SVG, smart urban development.

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

TABLE OF CONTENTS

Certificate 2
Acknowledgement 3
Abstract 4
Table of Contents 5
CHAPTER 1: INTRODUCTION 6
1.1 Project Overview 6
1.2 Problem Statement 6
1.3 Objectives 6
1.4 Scope of the Project 7
CHAPTER 2: BACKGROUND STUDY 8
2.1 Existing System 8
2.2 Need for the Proposed System 8
2.3 Technology Overview 8
CHAPTER 3: SYSTEM DESIGN AND METHODOLOGY 10
3.1 System Architecture 10
3.2 Working Flow 11
3.3 Main Modules and Features 11
3.4 OOP Concepts Used 12
3.5 Technologies and Tools 12
3.6 Hardware and Software Requirements 12
CHAPTER 4: IMPLEMENTATION AND RESULTS 13
4.1 Implementation Details 13
4.2 Important Source Code Explanations 13
4.3 Main Website Features 14
4.4 Project Screenshots 15
4.5 Testing and Observations 16
CHAPTER 5: CONCLUSION AND FUTURE SCOPE 18
5.1 Conclusion 18
5.2 Limitations 18
5.3 Future Improvements 18
REFERENCES 19

CHAPTER 1

INTRODUCTION

1.1 Project Overview

The Smart Urban Website, also named “Smart Urban Route Planning System” in the program, is a C++17 web application that helps a user find the quickest way to travel between two places in a small simulated city. The user selects a starting location and a destination in a web page. The program then calculates the best route, draws it on a map and shows the distance and the estimated travel time.

The project goes one step beyond a fixed shortest-path demo. Each road in the city has a traffic multiplier and can be closed, so the “best” route depends on the current road conditions. A Simulate Traffic button applies congestion to some roads and the route is recalculated automatically. A Reset Traffic button brings all roads back to normal.

The whole project is made of one C++ source file (main.cpp) and the third-party header httplib.h. The HTML, CSS and JavaScript of the web page are stored inside main.cpp as a string, so the compiled program can be run on its own and the page is opened at http://localhost:8080.

1.2 Problem Statement

In a city, the shortest road is not always the fastest road. Congestion and road closures can make a short road slow, and people may waste time if they only look at distance. A route planner for smart urban use should therefore consider travel time under current traffic conditions, show the result clearly and update it when the conditions change.

The problem handled in this project is: given a road network of a small city, a starting point, a destination and the present traffic situation, find the route with the minimum travel time and present it in a simple, visual form on a website.

1.3 Objectives

- To model a small city as a weighted graph using C++ data structures.
- To implement Dijkstra’s algorithm to find the minimum travel-time route.
- To include traffic congestion and road closure in the route calculation.

- To build a web server in C++ that exposes simple API endpoints and serves a web interface.
- To display the city map, the optimized route, journey details and road conditions in the browser.
- To validate user input and keep the shared data safe when several requests arrive.

1.4 Scope of the Project

The project covers a fixed city of 8 locations and 12 roads, route calculation between any two locations, a traffic simulation that changes the road conditions, and a browser interface to see the result. All data is kept in memory while the program is running.

The project does not use real traffic or map data, GPS, a database, user login or HTTPS. The city map and the traffic values are written directly in the source code, so the project is a learning model of a smart urban route planner and not a replacement for a real navigation service.

CHAPTER 2

BACKGROUND STUDY

2.1 Existing System

Today, people commonly use online map and navigation services to find routes. These services combine map data with live traffic information and suggest a route with an estimated time. They are very powerful, but they are large systems built by big companies and their internal working is hidden from the user. For a student, they are useful to use but not useful to learn how the route is actually calculated.

In academic programs, the shortest-path problem is usually taught with simple console programs. A typical program reads a graph, runs Dijkstra's algorithm and prints the shortest distance as text. Such programs usually use only distance as the weight, do not change with traffic and give no visual output.

2.2 Need for the Proposed System

A project that connects the algorithm to a visible, interactive result is easier to understand and to demonstrate. The proposed system fills this gap in the following ways:

- It uses travel time with a traffic multiplier instead of only distance, so a longer but free road can be chosen over a shorter congested road.
- It can handle a closed road and, if the closure splits the map, it reports that no route is available.
- It shows the city as a map in the browser, so the result can be understood at a glance.
- It needs no installation of a separate web server or database; running one executable is enough.
- Its complete source code is small enough to read and explain, which suits a C++ course project.

2.3 Technology Overview

The technologies that were actually used in the project are listed in Table 2.1.

Table 2.1: Technologies used in the project

Technology	Use in this project
C++17	Main programming language for the data model, algorithm, traffic logic and server code.
C++ Standard Library	vector, string, ostringstream, mutex and lock_guard, numeric_limits, reverse and iomanip are used in the program.
cpp-httplib 0.64.0	Header-only third-party library (MIT licence) used to run the HTTP server and handle GET requests.
HTML and CSS	Structure and styling of the web page, stored as a raw string inside main.cpp.
JavaScript	Fetches data from the API using fetch with async/await and updates the page.
SVG	Draws the city map (roads, locations, labels) in the browser.
JSON	Format of the server response. It is built manually in C++ using an output string stream.

Dijkstra's algorithm is the only algorithm used. It repeatedly selects the unvisited location with the smallest known travel time and updates the neighbours of that location. It works correctly here because all road weights are positive.

CHAPTER 3

SYSTEM DESIGN AND METHODOLOGY

3.1 System Architecture

The system follows a simple client–server design. The C++ program is the server, and the web browser is the client. The server keeps the city data and does all calculations. The browser only shows the data and sends requests when the user presses a button.

Table 3.1: Layers of the system

Layer	Description
Presentation layer	HTML, CSS and JavaScript page served at "/". It draws the SVG map and the information panels from the JSON received.
API layer	Handlers written with cpp-httplib for /api/state, /api/route, /api/simulate and /api/reset. Every handler returns the full state as JSON.
Logic layer	Functions resetTraffic(), simulateTraffic(), calculateRoute() and getStateJSON().
Data layer	In-memory global vectors cities and roads, plus variables for source, destination, route, distance and time.

The city graph has 8 locations (nodes) and 12 roads (edges), as given in Table 3.2 and Table 3.3. The graph is undirected, so every road can be used in both directions.

Table 3.2: Locations in the city (ID: name)

ID	Location	ID	Location
0	Central Station	4	Airport
1	City Hospital	5	Tech Park
2	University	6	Bus Terminal
3	Shopping Mall	7	Old Town

Table 3.3: Roads in the city (distance in km, base time in minutes)

No.	Road	Km	Time (min)	Road index
1	Central Station – City Hospital	4	8	0
2	Central Station – Bus Terminal	3	7	1
3	Central Station – Old Town	2.5	6	2
4	City Hospital – University	5	9	3
5	City Hospital – Bus Terminal	4	8	4
6	University – Shopping Mall	3.5	7	5

No.	Road	Km	Time (min)	Road index
7	University – Tech Park	4.5	9	6
8	Shopping Mall – Airport	6	11	7
9	Shopping Mall – Tech Park	4	8	8
10	Airport – Tech Park	5.5	10	9
11	Tech Park – Bus Terminal	5	9	10
12	Bus Terminal – Old Town	3	6	11

3.2 Working Flow

The working of the system from start to finish is as follows:

- The program starts, creates a httplib::Server object, registers the handlers and listens on 127.0.0.1, port 8080.
- The browser opens http://localhost:8080 and receives the web page. The page calls /api/state to get the current data.
- The JavaScript fills the two drop-down lists with the 8 locations and draws the map.
- When the user presses Find Best Route, the page calls /api/route with the selected start and end IDs. The server checks the IDs, stores them, runs Dijkstra's algorithm and returns the new state.
- When the user presses Simulate Traffic, the page calls /api/simulate. The server changes the traffic values, recalculates the route for the same source and destination and returns the state.
- When the user presses Reset Traffic, the page calls /api/reset. All roads return to normal and the route is recalculated.
- After every response the page redraws the map, the route details, the journey information and the road conditions panel.

3.3 Main Modules and Features

Table 3.4: Modules of the project

Module	Purpose
Data model	struct City and struct Road, and the vectors cities and roads that hold the map.
Route calculation	calculateRoute() runs Dijkstra's algorithm, rebuilds the path and adds up the distance.

Module	Purpose
Traffic management	resetTraffic() and simulateTraffic() set the traffic multipliers and road status.
JSON builder	getStateJSON() creates the response sent to the browser.
Web server	main() registers the handlers and starts the server.
Web interface	Embedded HTML, CSS and JavaScript: SVG map, route box, journey information, road conditions and buttons.

3.4 OOP Concepts Used

This project is written mostly in a procedural style, with global data and functions. It does not define its own classes, so the OOP concepts have to be described honestly according to what is present in the code:

- **Structures and objects:** City and Road are user-defined structures that group related data (name and coordinates; end points, distance, time, traffic and open status). Objects of these types are stored in vectors. The program also creates objects of library classes such as httplib::Server, vector, string, mutex and ostringstream and uses their member functions.
- **Abstraction and reuse of classes:** The program uses the HTTP server through the public interface of httplib::Server (Get() and listen()) without knowing how sockets are handled inside it.
- **Encapsulation (limited):** Access to the shared data is controlled by locking dataMutex with lock_guard in every handler. The data itself is global and public, so full encapsulation with private members is not applied.
- **Lambda expressions:** Each API handler is a lambda function passed to server.Get().
- **Inheritance and polymorphism:** These are not used in this project. They are possible future improvements, for example by moving the code into classes such as Graph, Road and TrafficManager (see Chapter 5).

3.5 Technologies and Tools

The program is compiled with a C++17 compiler (g++, clang++ or MSVC) together with httplib.h placed in the same folder. On Linux and macOS the flag -pthread is needed, and on Windows with MinGW the library ws2_32 is linked. The page is viewed in any modern web browser. Visual Studio Code or any text editor can be used to edit the source.

3.6 Hardware and Software Requirements

Table 3.5: Requirements

Item	Requirement
Hardware	Any computer able to run a C++ compiler and a web browser (no special hardware needed).
Operating system	Windows 10 or later, Linux or macOS.
Compiler	g++, clang++ or MSVC with C++17 support.
Library	cpp-httplib (httplib.h), version 0.64.0 was used.
Browser	A modern browser with JavaScript and SVG support.
Network	Only the local machine is used; the server listens on 127.0.0.1:8080.

CHAPTER 4

IMPLEMENTATION AND RESULTS

4.1 Implementation Details

The complete application is in main.cpp, which has about 1,195 lines. About 1,000 of these lines are the embedded web page and the rest is the C++ logic and server. The file is organised in this order: data structures and global data, helper functions, traffic functions, Dijkstra's algorithm, the JSON builder, the web page string and finally main().

The map is stored in two vectors. The first vector holds the 8 City objects with their names and screen coordinates for the SVG map. The second vector holds the 12 Road objects:

```
struct City { string name; int x, y; };
struct Road { int u, v; double km, time, traffic; bool open; };

// example roads: {from, to, km, base time, traffic, open}
{0, 1, 4, 8, 1, true}   // Central Station - City Hospital
{0, 6, 3, 7, 1, true}   // Central Station - Bus Terminal
```

4.2 Important Source Code Explanations

4.2.1 Dijkstra's algorithm – calculateRoute()

The function keeps three arrays: dist (best known time to each location), previous (the location before it on the best path) and visited. In each step it chooses the unvisited location with the smallest dist value, marks it visited and checks all open roads connected to it. The weight used for a road is time × traffic. This is an array-based version, so its time complexity is $O(N^2)$ plus a loop over the roads for each step, which is fine for 8 locations.

```
double weight = roads[r].time * roads[r].traffic;
double newTime = dist[u] + weight;
if (newTime < dist[v]) {
    dist[v] = newTime;
    previous[v] = u;
}
```

After the loop, the path is rebuilt backwards from the destination using the previous array and then reversed. The total distance is calculated by adding the km value of every road on the path using roadIndex(). The total time is dist[destination]. If the destination was never reached, found stays false and the page shows that no route is available.

4.2.2 Traffic simulation – simulateTraffic()

This function first calls resetTraffic(), then increases the scenario counter and sets the traffic multipliers of selected roads. Scenario 1 gives congestion on City Hospital – University (2.2×), University – Shopping Mall (1.8×) and Shopping Mall – Tech Park (1.7×). The code also contains branches for scenario 2 (several congested roads and the closed Shopping Mall – Airport road) and scenario 3. See Section 4.5 for an observation about how the counter behaves.

4.2.3 API handlers and validation

Each handler is a lambda passed to server.Get(). The /api/route handler checks that both parameters are present, converts them with stoi() inside a try–catch block and checks that the IDs are between 0 and 7. In all error cases it returns HTTP status 400 with a JSON error message. Handlers that read or change data first create a lock_guard<mutex> on dataMutex.

```
lock_guard<mutex> lock(dataMutex);
source = s;
destination = d;
res.set_content(getStateJSON(), "application/json");
```

4.2.4 JSON response and the web page

getStateJSON() calls calculateRoute() and then writes found, source, destination, distance, time, scenario, route, cities and roads into an ostringstream. In the browser, loadState(), findRoute(), simulateTraffic() and resetTraffic() call the API using fetch with async/await. drawMap() builds the SVG elements: blue lines for the route, orange for congested roads, red dashed for closed roads and grey for normal roads. escapeHTML() is used before location names are put into the page.

Table 4.1: API endpoints

Endpoint	Description
GET /	Returns the web interface.
GET /api/state	Returns the current route, cities and road conditions.
GET /api/route?start=&end=	Sets the source and destination and returns the new route. Returns 400 for missing, non-numeric or out-of-range IDs.
GET /api/simulate	Applies the traffic simulation and returns the new state.
GET /api/reset	Clears all congestion and closures and returns the state.

4.3 Main Website Features

- Interactive SVG map of the city with location names and the distance of each road.
- Drop-down lists to choose the starting location and the destination.
- Find Best Route, Simulate Traffic and Reset Traffic buttons.
- Optimized route details with the stop-by-stop path, total distance and travel time.
- Journey information cards showing distance in km and time in minutes.
- Road conditions panel that lists congested and closed roads.
- Colour legend for the route, congestion, closed and normal roads, and a display of the current scenario number.

4.4 Project Screenshots

The following screenshots were taken from the running project in a web browser. Figure 4.1 shows the home page when the server starts. The default selection is Central Station to Airport, and the route found is Central Station → Bus Terminal → Tech Park → Airport.

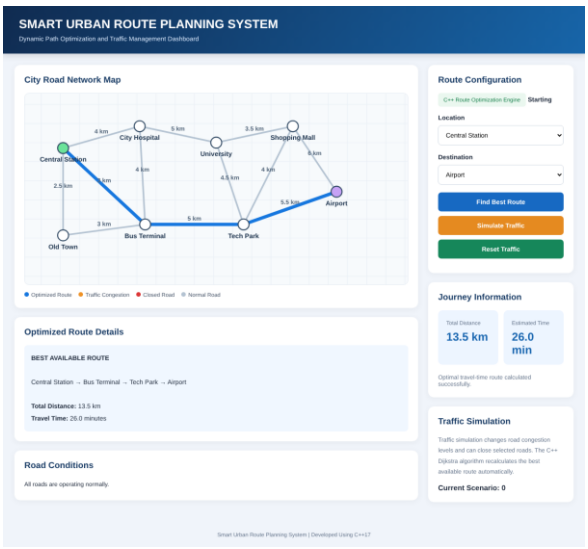


Figure 4.1: Home page with the default route (Central Station to Airport)

Figure 4.2 shows a route from University to Old Town in normal traffic. The program chose University → City Hospital → Central Station → Old Town, with 11.5 km and 23.0 minutes.

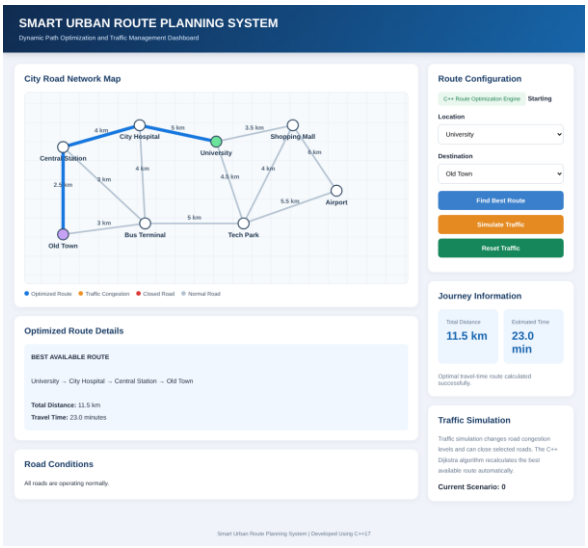


Figure 4.2: Route from University to Old Town with normal traffic

Figure 4.3 shows the same journey after pressing Simulate Traffic. Three roads became congested (orange), so the route changed to University → Tech Park → Bus Terminal → Old Town (12.5 km, 24.0 minutes). The longer route is chosen because it is faster. The Road Conditions panel lists the congested roads.

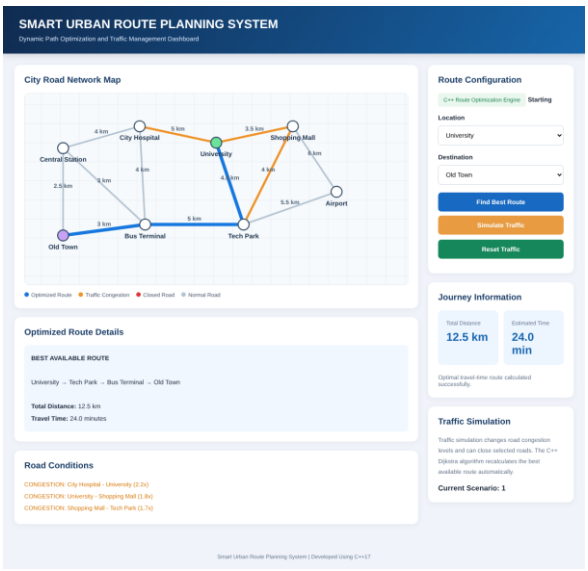


Figure 4.3: Route recalculated after Simulate Traffic

4.5 Testing and Observations

The project was compiled with g++ 13.3 using -std=c++17 and run on a Linux system. The API was tested with curl, and the web page was operated with an automated Chromium browser, which also produced the screenshots above. No separate test framework was used. The results are given in Table 4.2.

Table 4.2: Test cases and observed results

No.	Test	Observed result	Status
1	Compile with g++ -std=c++17 -pthread	Built without errors; executable created.	Pass
2	Open / in the browser	HTTP 200; page and map displayed.	Pass
3	Default route 0 → 4, normal traffic	Route [0, 6, 5, 4]; 13.5 km; 26.0 min.	Pass
4	Route 2 → 7, normal traffic	Route [2, 1, 0, 7]; 11.5 km; 23.0 min.	Pass
5	Route 2 → 7 after Simulate Traffic	Route [2, 5, 6, 7]; 12.5 km; 24.0 min (route changed).	Pass
6	Route 7 → 3 after Simulate Traffic	Same route [7, 6, 5, 3]; time rises from 23.0 to 28.6 min.	Pass
7	Same start and destination (3 → 3)	Route [3]; 0 km; 0 min.	Pass
8	Reset Traffic	Scenario returns to 0; traffic values normal.	Pass
9	Invalid input: start=abc, end=9, missing end, start=-1	HTTP 400 returned in all four cases.	Pass
10	Press Simulate Traffic three times	Scenario stays 1 every time (see below).	Observation

Observation: In simulateTraffic(), the call to resetTraffic() sets the scenario counter back to 0 just before it is increased. Because of this, the scenario number is always 1, and the code for scenario 2 (including the road closure) and scenario 3 is not reached when the buttons are used. The README of the project describes a cycle of three scenarios, so this is a bug in the submitted code and not the intended behaviour. The closed-road handling inside calculateRoute() (skipping roads whose open value is false) is present in the code but could not be tested through the website for this reason. The fix is described in Chapter 5.

The tests also show that the route time in the result already includes the traffic multipliers, so the same route can show a different time after congestion is applied (test 6).

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

The Smart Urban Website project achieved its main objective of finding and showing the fastest route in a simulated city. A weighted graph of 8 locations and 12 roads was built in C++, and Dijkstra's algorithm was used with traffic-adjusted travel times. The C++ program works as its own web server using cpp-httplib, and the embedded web page shows the map, route, distance, time and road conditions. Tests showed that the route changes when congestion is applied and that invalid requests are rejected with HTTP 400.

While making the project I learned how a graph can be stored with structures and vectors, how Dijkstra's algorithm works step by step, how a C++ program can respond to HTTP requests, and why shared data needs a mutex.

5.2 Limitations

- The city map and traffic values are fixed in the source code; no real traffic or map data is used.
- Due to the scenario-counter problem, only the first traffic scenario is applied and road closure cannot be triggered from the website.
- There are no user-defined classes, so OOP features such as inheritance and polymorphism are not used.
- All data is stored in memory and is lost when the server stops; there is no database.
- There is no user login, no HTTPS, and the server accepts connections only from the same computer.
- The shortest-path code is an $O(N^2)$ version, suitable for a small map only. Some helper code (number(), isRouteRoad(), <queue>) is unused. Testing was manual.

5.3 Future Improvements

- Fix simulateTraffic() so that the scenario counter is not reset on every call, which will make the three scenarios rotate and allow the closed-road case to be tested.

- Reorganise the code into classes such as Graph, Road and TrafficManager, which will allow encapsulation and, if needed, inheritance and polymorphism.
- Load the city and road data from a file or a database instead of fixing it in the code.
- Add automated tests for the algorithm and the API, and use a priority queue in Dijkstra's algorithm for larger maps.

REFERENCES

[1] Yuji Hirose, “cpp-httplib: A C++ header-only HTTP/HTTPS server and client library” (version 0.64.0, MIT licence). https://github.com/yhirose/cpp-httplib

[2] E. W. Dijkstra, “A note on two problems in connexion with graphs”, Numerische Mathematik, vol. 1, pp. 269–271, 1959.

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 3rd ed., MIT Press, 2009 (chapter on single-source shortest paths).

[4] B. Stroustrup, The C++ Programming Language, 4th ed., Addison-Wesley, 2013.

[5] cppreference.com, “C++ reference” (std::vector, std::mutex, std::lock_guard, std::ostringstream, std::numeric_limits). https://en.cppreference.com

[6] MDN Web Docs, “Using the Fetch API” and “SVG: Scalable Vector Graphics”. https://developer.mozilla.org

[7] Project source files: main.cpp, httplib.h and README.md of the Smart Urban Route Planning System.